

Content Plagiarism Testing (NLP Project)

In [14]:

```
#Import required dependencies:
```

```
import warnings
```

```
warnings.filterwarnings("ignore") #ignore all incoming warnings (if any)
```

```
import nltk
```

```
nltk.download("popular")
```

```
import pandas as pd
```

```
import string
```

```
from nltk.corpus import stopwords
```

```
import joblib
```

```
import numpy as np
```

```
[nltk_data] Downloading collection 'popular'
[nltk_data] |
[nltk_data] | Downloading package cmudict to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package cmudict is already up-to-date!
[nltk_data] | Downloading package gazetteers to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package gazetteers is already up-to-date!
[nltk_data] | Downloading package genesis to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package genesis is already up-to-date!
[nltk_data] | Downloading package gutenbergt to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package gutenbergt is already up-to-date!
[nltk_data] | Downloading package inaugural to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package inaugural is already up-to-date!
[nltk_data] | Downloading package movie_reviews to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package movie_reviews is already up-to-date!
[nltk_data] | Downloading package names to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package names is already up-to-date!
[nltk_data] | Downloading package shakespear to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package shakespear is already up-to-date!
[nltk_data] | Downloading package stopwords to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package stopwords is already up-to-date!
[nltk_data] | Downloading package treebank to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package treebank is already up-to-date!
[nltk_data] | Downloading package twitter_samples to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package twitter_samples is already up-to-date!
[nltk_data] | Downloading package omw to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package omw is already up-to-date!
[nltk_data] | Downloading package omw-1.4 to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package omw-1.4 is already up-to-date!
[nltk_data] | Downloading package wordnet to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package wordnet is already up-to-date!
[nltk_data] | Downloading package wordnet2021 to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package wordnet2021 is already up-to-date!
[nltk_data] | Downloading package wordnet31 to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
```

```

[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package wordnet31 is already up-to-date!
[nltk_data] | Downloading package wordnet_ic to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package wordnet_ic is already up-to-date!
[nltk_data] | Downloading package words to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package words is already up-to-date!
[nltk_data] | Downloading package maxent_ne_chunker to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package maxent_ne_chunker is already up-to-date!
[nltk_data] | Downloading package punkt to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package punkt is already up-to-date!
[nltk_data] | Downloading package snowball_data to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package snowball_data is already up-to-date!
[nltk_data] | Downloading package averaged_perceptron_tagger to
[nltk_data] | C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data] | Package averaged_perceptron_tagger is already up-
[nltk_data] | to-date!
[nltk_data] |
[nltk_data] Done downloading collection popular

```

In [15]:

```

#Import the dataset:

data = pd.read_csv("dataset.csv")
data.head()

```

Out[15]:

Unnamed: 0		source_text	plagiarized_text	label
0	0	Researchers have discovered a new species of b...	Scientists have found a previously unknown but...	1
1	1	The moon orbits the Earth in approximately 27....	Our natural satellite takes around 27.3 days t...	1
2	2	Water is composed of two hydrogen atoms and on...	H2O consists of 2 hydrogen atoms and 1 oxygen ...	1
3	3	The history of Rome dates back to 753 BC.	Rome has a long history that can be traced bac...	1
4	4	Pluto was once considered the ninth planet in ...	In the past, Pluto was classified as the ninth...	1

In [16]:

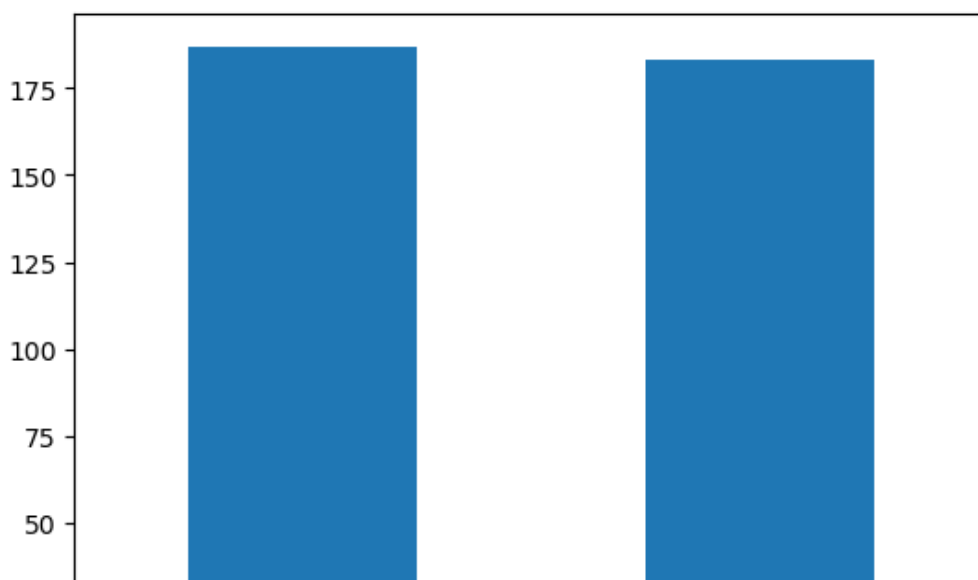
```

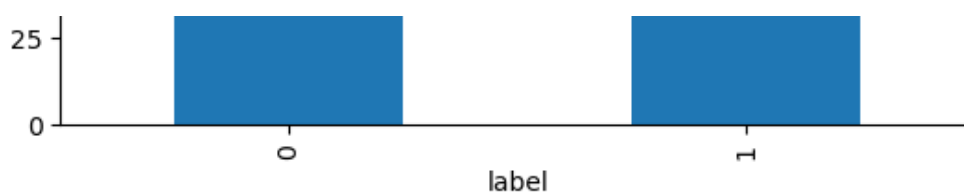
#Check for distribution in the target variable:
data["label"].value_counts().plot(kind="bar")

```

Out[16]:

<Axes: xlabel='label'>





In [17]:

```
data["label"].value_counts()
```

Out[17]:

```
label
0    187
1    183
Name: count, dtype: int64
```

In [18]:

```
#Check for value-structures in both the input fields:
```

```
for i in range(5):
    print(f"Row {i+1} SOURCE TEXT:      ",data["source_text"].values[i])
    print(f"Row {i+1} PLAGIARIZED TEXT:  ",data["plagiarized_text"].values[i])

    print("\n\n\n")
```

Row 1 SOURCE TEXT:	Researchers have discovered a new species of butterfly in the Amazon rainforest.
Row 1 PLAGIARIZED TEXT:	Scientists have found a previously unknown butterfly species in the Amazon jungle.

Row 2 SOURCE TEXT:	The moon orbits the Earth in approximately 27.3 days.
Row 2 PLAGIARIZED TEXT:	Our natural satellite takes around 27.3 days to complete one orbit around our planet.

Row 3 SOURCE TEXT:	Water is composed of two hydrogen atoms and one oxygen atom.
Row 3 PLAGIARIZED TEXT:	H2O consists of 2 hydrogen atoms and 1 oxygen atom.

Row 4 SOURCE TEXT:	The history of Rome dates back to 753 BC.
Row 4 PLAGIARIZED TEXT:	Rome has a long history that can be traced back to 753 BC.

Row 5 SOURCE TEXT:	Pluto was once considered the ninth planet in our solar system.
Row 5 PLAGIARIZED TEXT:	In the past, Pluto was classified as the ninth planet in our sun's planetary system.

In [19]:

```
#Remove all punctuation marks in both input features:
```

```
def preprocess_text(text):
    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))
    # Convert to lowercase
```

```

text = text.lower()
# Remove stop words
stop_words = set(stopwords.words("english"))
text = " ".join(word for word in text.split() if word not in stop_words)
return text

```

```

data["source_text"] = data["source_text"].apply(preprocess_text)
data["plagiarized_text"] = data["plagiarized_text"].apply(preprocess_text)

```

In [20]:

```
#Check again for value-structures in both the input fields:
```

```

for i in range(5):
    print(f"Row {i+1} SOURCE TEXT:           ",data["source_text"].values[i])
    print(f"Row {i+1} PLAGIARIZED TEXT:      ",data["plagiarized_text"].values[i])

    print("\n\n\n")

```

```

Row 1 SOURCE TEXT:           researchers discovered new species butterfly amazon rainforests

```

```

Row 1 PLAGIARIZED TEXT:      scientists found previously unknown butterfly species amazon
jungle

```

```

Row 2 SOURCE TEXT:           moon orbits earth approximately 273 days

```

```

Row 2 PLAGIARIZED TEXT:      natural satellite takes around 273 days complete one orbit around planet

```

```

Row 3 SOURCE TEXT:           water composed two hydrogen atoms one oxygen atom

```

```

Row 3 PLAGIARIZED TEXT:      h2o consists 2 hydrogen atoms 1 oxygen atom

```

```

Row 4 SOURCE TEXT:           history rome dates back 753 bc

```

```

Row 4 PLAGIARIZED TEXT:      rome long history traced back 753 bc

```

```

Row 5 SOURCE TEXT:           pluto considered ninth planet solar system

```

```

Row 5 PLAGIARIZED TEXT:      past pluto classified ninth planet suns planetary system

```

In []:

In [21]:

```
# Design ML pipelines to convert both the input features into vector form and then pass it on to their respective ml model:
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.model_selection import train_test_split , cross_validate , StratifiedKFold
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.ensemble import ExtraTreesClassifier , AdaBoostClassifier , RandomForestClassifier
```

```

from xgboost import XGBClassifier

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    r2_score,
    mean_squared_error,
    mean_absolute_error
)

```

```

X = data["source_text"]+data["plagiarized_text"]
y = data["label"]

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

#All classification pipelines:

```

pipelines = {
    "Logistic Regression": Pipeline([
        ("vectorizer", TfidfVectorizer()),
        ("model", LogisticRegression(max_iter=1000, random_state=42))
    ]),

    "KNN": Pipeline([
        ("vectorizer", TfidfVectorizer()),
        ("model", KNeighborsClassifier(n_neighbors=7, n_jobs=-1))
    ]),

    "Random Forest": Pipeline([
        ("vectorizer", TfidfVectorizer()),
        ("model", RandomForestClassifier(
            random_state=42,
            n_jobs=-1
        ))
    ]),

    "Extra Trees": Pipeline([
        ("vectorizer", TfidfVectorizer()),
        ("model", ExtraTreesClassifier(
            n_estimators=120,
            max_depth=9,
            random_state=42,
            n_jobs=-1
        ))
    ]),

    "AdaBoost": Pipeline([
        ("vectorizer", TfidfVectorizer()),
        ("model", AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=4),

```

```

        n_estimators=200,
        learning_rate=0.8,
        random_state=42
    ))
]),

"XGBoost": Pipeline([
    ("vectorizer", TfidfVectorizer()),
    ("model", XGBClassifier(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1,
        tree_method="hist",
        eval_metric="logloss"
    ))
])
}

```

In [22]:

```

# Cross Validation:

## Model Dictionary:
models = {
    "Logistic Regression": pipelines["Logistic Regression"],
    "KNN": pipelines["KNN"],
    "Random Forest": pipelines["Random Forest"],
    "Extra Trees": pipelines["Extra Trees"],
    "AdaBoost": pipelines["AdaBoost"],
    "XGBoost": pipelines["XGBoost"]
}

skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42) # 3 folds = faster

scoring = ['accuracy', 'f1_weighted']

results = []

for name, model in models.items():
    print(f"Running {name}...")

    cv_results = cross_validate(
        estimator=model,
        X=X_train,
        y=y_train,          # Use y_train (or y_train_encoded if you actually created it)
        cv=skf,
        scoring=scoring,
        n_jobs=-1
    )

    results.append({
        "Model": name,
        "CV Accuracy": round(np.mean(cv_results['test_accuracy']), 4),
        "CV F1 Score": round(np.mean(cv_results['test_f1_weighted']), 4)
    })

results_df = pd.DataFrame(results).sort_values(by="CV Accuracy", ascending=False)
print("\n" + "="*50)
print(results_df)

```

Running Logistic Regression...

Running KNN...

Running Random Forest...

Running Extra Trees...

Running AdaBoost...
Running XGBoost...

```
=====
```

	Model	CV Accuracy	CV F1 Score
0	Logistic Regression	0.8850	0.8848
4	AdaBoost	0.8345	0.8313
1	KNN	0.8176	0.8156
2	Random Forest	0.8109	0.8030
3	Extra Trees	0.7973	0.7921
5	XGBoost	0.6722	0.6324

In [23]:

```
#Make predictions using the best model:
```

```
best_model_name = results_df.iloc[0]["Model"]
best_pipeline = pipelines[best_model_name]

best_pipeline.fit(X_train, y_train)
y_pred = best_pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
#Export the best performing model:
```

```
joblib.dump(best_pipeline, "Content_Plagiarism_Detection_Model.pkl")
```

Accuracy: 0.8918918918918919
Precision: 0.8918918918918919
Recall: 0.8918918918918919
F1 Score: 0.8918918918918919

Out[23]:

```
['Content_Plagiarism_Detection_Model.pkl']
```

In [24]:

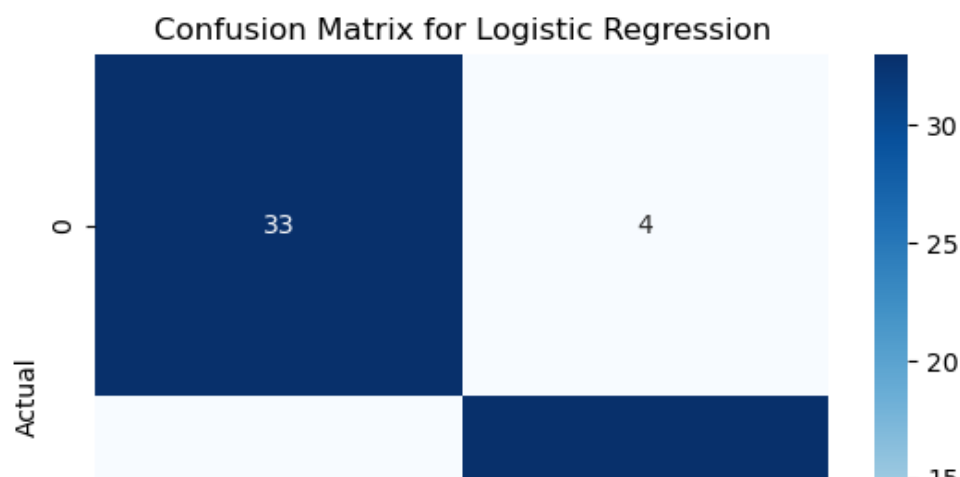
```
cm = confusion_matrix(y_test, y_pred)

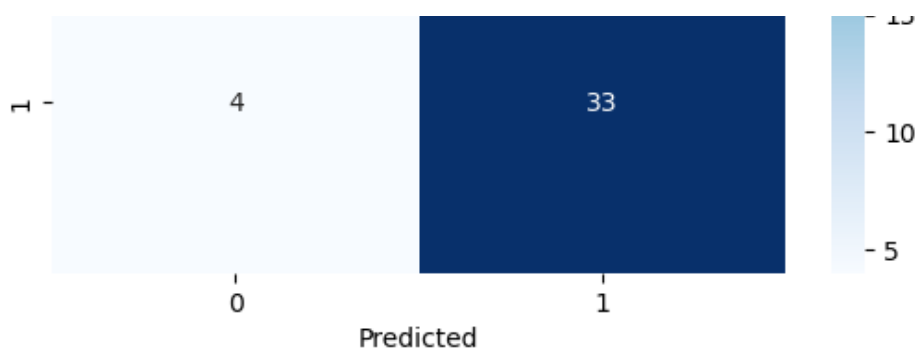
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix for Logistic Regression")

plt.show()
```





In [25]:

```
print("True Positives:", cm[0][0])
print("False Positives:", cm[0][1])
print("True Negatives:", cm[1][1])
print("False Negatives:", cm[1][0])
```

True Positives: 33
False Positives: 4
True Negatives: 33
False Negatives: 4

In [26]:

#Now build the plagiarism detector function:

```
def detect(sen):

    result = best_pipeline.predict([sen])

    if result[0] == 1:
        print("Plagiarized text")
    else:
        print("No Plagiarism found")
```

```
detect("My name is Om Satyawar Pathak")
detect("Today's weather is not good.")
detect("Gravity is the force that attracts two bodies towards each other.")
detect("This is a unique ,and original sentence")
```

Plagiarized text
No Plagiarism found
Plagiarized text
No Plagiarism found

In []:

PROJECT BY:

- **Name:** Om Satyawar Pathak
- **Email:** omsatyawanpathakgit@gmail.com (or) omsatyawanpathakwebdevelopment@gmail.com
- **LinkedIn:** www.linkedin.com/in/om-satyawan-pathak-029b02368

In []: